

```

import torch
import torch.nn as nn
import torch.nn.functional as F
from torch_geometric.data import HeteroData
from torch_geometric.nn import GATConv
from torch_geometric.loader import DataLoader
from transformers import AutoTokenizer, AutoModel
from peft import LoraConfig, get_peft_model # pip install peft

# -----
# 1) DATA PREPARATION
# -----

def prepare_heterodata(email_records, sender_encoder, recipient_encoder, tokenizer, max_len=128):
    """
    email_records: list of dicts with keys
    - 'sender_id', 'recipient_id'
    - 'body_text' (string)
    - 'hour', 'day', 'mailbox_id' (floats/ints)
    Returns a HeteroData with:
    - node features: just indices for senders/recipients
    - edge_index & edge_attr (float tensor)
    - tokenized inputs stored for IG at inference
    """

    # encode text
    toks = tokenizer(
        [e['body_text'] for e in email_records],
        padding='max_length', truncation=True, max_length=max_len,
        return_tensors='pt'
    )
    input_ids = toks.input_ids # [E, max_len]
    attention_mask = toks.attention_mask # [E, max_len]

    # numerical edge features
    edge_meta = torch.stack([
        torch.tensor([e['hour'], e['day'], e['mailbox_id']], dtype=torch.float)
        for e in email_records
    ], dim=0) # [E,3]

    # label-encode nodes
    senders = torch.tensor([sender_encoder[e['sender_id']] for e in email_records], dtype=torch.long)
    recips = torch.tensor([recipient_encoder[e['recipient_id']] for e in email_records], dtype=torch.long)

    # build HeteroData
    data = HeteroData()
    data['sender'].node_id = torch.arange(len(sender_encoder)) # assume contiguous
    data['recipient'].node_id = torch.arange(len(recipient_encoder))

    data['sender', 'emails', 'recipient'].edge_index = torch.stack([senders, recips], dim=0)
    # we'll store a placeholder for edge_attr; actual input to model will concat text_emb+edge_meta
    data['sender', 'emails', 'recipient'].edge_attr = edge_meta

    # stash token inputs for inference
    data['sender', 'emails', 'recipient'].input_ids = input_ids
    data['sender', 'emails', 'recipient'].attention_mask = attention_mask

    return data

```

```

# -----
# 2) MODEL ARCHITECTURE
# -----
class HybridVGAE(nn.Module):
    def __init__(self,
                  num_senders, num_recipients,
                  edge_meta_dim=3,
                  gnn_emb=128,
                  adapter_r=8,
                  device='cuda'):
        super().__init__()
        self.device = device

        # ---- In-loop text encoder with LoRA adapters ----
        base = AutoModel.from_pretrained("BAAI/bge-small-en")
        # attach LoRA
        lora_cfg = LoraConfig(
            r=adapter_r, lora_alpha=32, target_modules=['query', 'value'],
            lora_dropout=0.05, bias="none"
        )
        self.text_encoder = get_peft_model(base, lora_cfg).to(device)

        # project CLS(384) → GNN embedding
        self.text_proj = nn.Linear(base.config.hidden_size, gnn_emb)

        # project edge_meta → gnn_emb
        self.meta_proj = nn.Linear(edge_meta_dim, gnn_emb)

        # node embeddings
        self.sender_emb = nn.Embedding(num_senders, gnn_emb)
        self.recipient_emb = nn.Embedding(num_recipients, gnn_emb)

        # VGAE encoders for mu & logvar
        self.conv_mu = GATConv((-1, -1), gnn_emb, edge_dim=gnn_emb)
        self.conv_lv = GATConv((-1, -1), gnn_emb, edge_dim=gnn_emb)

        # MLP to fuse into per-edge z
        self.edge_mlp = nn.Sequential(
            nn.Linear(3*gnn_emb, gnn_emb), nn.ReLU(),
            nn.Linear(gnn_emb, gnn_emb)
        )

        # decoder: reconstruct meta+text features
        self.decoder = nn.Sequential(
            nn.Linear(gnn_emb, gnn_emb), nn.ReLU(),
            nn.Linear(gnn_emb, edge_meta_dim + base.config.hidden_size)
        )

        # contrastive projection head
        self.proj_head = nn.Sequential(
            nn.Linear(gnn_emb, gnn_emb), nn.ReLU(),
            nn.Linear(gnn_emb, 64)
        )

        # register a fixed prior center (zero)
        self.register_buffer('prior_mean', torch.zeros(gnn_emb))
        self.register_buffer('prior_var', torch.ones(gnn_emb))

```

```

def encode(self, data, mask_rate=0.3):
    # 1) Text embeddings
    ids = data['sender','emails','recipient'].input_ids.to(self.device)
    att = data['sender','emails','recipient'].attention_mask.to(self.device)
    # frozen except adapters
    out = self.text_encoder(input_ids=ids, attention_mask=att, return_dict=True)
    cls = out.last_hidden_state[:,0,:] # [E,384]
    t_emb = self.text_proj(cls) # [E,gnn_emb]

    # 2) Meta embeddings
    meta = data['sender','emails','recipient'].edge_attr.to(self.device)
    m_emb = self.meta_proj(meta) # [E,gnn_emb]

    # 3) Node embeddings
    s_idx = data['sender','emails','recipient'].edge_index[0].to(self.device)
    r_idx = data['sender','emails','recipient'].edge_index[1].to(self.device)
    s = self.sender_emb(s_idx) # [E,gnn_emb]
    r = self.recipient_emb(r_idx) # [E,gnn_emb]

    # 4) Optional Masked-Edge MAE: mask a random subset of features
    mask = (torch.rand_like(m_emb) < mask_rate).float()
    m_masked = m_emb * (1-mask)

    # 5) Fuse: [s, r, t_emb] → messages for graph conv
    # and also form initial edge_input for edge-mlp
    edge_feat = t_emb + m_masked # you can also concat + project
    z_mu = self.conv_mu((s, r), data['sender','emails','recipient'].edge_index, edge_attr=edge_feat)
    z_lv = self.conv_lv((s, r), data['sender','emails','recipient'].edge_index, edge_attr=edge_feat)

    # reparameterize
    std = torch.exp(0.5 * z_lv)
    eps = torch.randn_like(std)
    z = z_mu + eps * std # [E,gnn_emb]

    # final edge embedding
    edge_input = torch.cat([s, r, z], dim=1)
    z_final = self.edge_mlp(edge_input) # [E,gnn_emb]

    return z_final, z_mu, z_lv

def decode(self, z_final):
    # reconstruct [meta_dim + text_emb_dim]
    return self.decoder(z_final) # [E, edge_meta_dim+384]

def forward(self, data, mask_rate=0.3):
    z_final, mu, lv = self.encode(data, mask_rate)
    recon = self.decode(z_final)

    # Contrastive projections
    proj = self.proj_head(z_final) # [E,64]
    return recon, mu, lv, proj

```

```

# -----

```

```

# 3) TRAINING LOOP

```

```

# -----

```

```

def train_unsupervised(model, data, optimizer,

```

```

        epochs=50,
        beta=0.01,
        tau=0.5,
        contrastive_weight=1.0,
        device='cuda'):
"""
- beta: weight for KL term
- tau: temperature for InfoNCE
"""

model.to(device)
E = data['sender','emails','recipient'].edge_index.size(1)

# Precompute all-augmented pair indices
idx = torch.arange(E).to(device)

for epoch in range(1, epochs+1):
    model.train()
    optimizer.zero_grad()

    # ---- Forward pass ----
    recon, mu, lv, proj = model(data, mask_rate=0.3)

    # 1) Reconstruction + KL (ELBO)
    # ground truth: concat [meta, text_emb]
    meta = data['sender','emails','recipient'].edge_attr.to(device)
    cls = model.text_encoder(
        input_ids=data['sender','emails','recipient'].input_ids.to(device),
        attention_mask=data['sender','emails','recipient'].attention_mask.to(device),
        return_dict=True
    ).last_hidden_state[:,0,:]
    target = torch.cat([meta, cls], dim=1)
    recon_loss = F.mse_loss(recon, target, reduction='mean')

    # KL per-edge
    kl = -0.5 * torch.sum(1 + lv - mu.pow(2) - lv.exp(), dim=1).mean()

    # 2) Contrastive InfoNCE on proj
    # build two augmentations by random shuffling + masking
    # Here: simple in-batch negatives
    sim = torch.matmul(proj, proj.T) / tau # [E,E]
    labels = torch.arange(E).to(device)
    contrastive_loss = F.cross_entropy(sim, labels)

    # total loss
    loss = recon_loss + beta*kl + contrastive_weight*contrastive_loss
    loss.backward()
    optimizer.step()

    if epoch % 10 == 0:
        print(f"Epoch {epoch:03d} | Recon={recon_loss:.4f} | KL={kl:.4f} | Contrast={contrastive_loss:.4f}")

# -----
# 4) INFERENCE & VISUALIZATION
# -----
@torch.no_grad()
def detect_anomalies(model, data, sender_map, top_pct=0.03, device='cuda'):
    model.eval()

```

```

recon, mu, lv, _ = model(data, mask_rate=0.0)
# compute per-edge ELBO score = rec_err + beta*kl
meta = data['sender','emails','recipient'].edge_attr.to(device)
cls = model.text_encoder(
    input_ids=data['sender','emails','recipient'].input_ids.to(device),
    attention_mask=data['sender','emails','recipient'].attention_mask.to(device),
    return_dict=True
).last_hidden_state[:,0,:]
target = torch.cat([meta, cls], dim=1)
rec_err = F.mse_loss(recon, target, reduction='none').mean(dim=1)

kl_per_edge = -0.5 * torch.sum(1 + lv - mu.pow(2) - lv.exp(), dim=1)

score = rec_err + kl_per_edge # higher = more anomalous

# dynamic per-sender threshold
edge_idx = data['sender','emails','recipient'].edge_index
senders = edge_idx[0].cpu().numpy()
scores = score.cpu().numpy()

flags = []
for s in np.unique(senders):
    idxs = np.where(senders==s)[0]
    thr = np.quantile(scores[idxs], 1 - top_pct)
    flags.extend(idxs[scores[idxs] > thr])

return sorted(flags), scores

def visualize_flags(flags, data, num=10):
    print(f"Top {num} flagged edges:")
    for idx in flags[:num]:
        txt = data['sender','emails','recipient'].input_ids[idx]
        print(f"Edge {idx}: text_tokens={txt[:10].tolist()} ...")

```